ETH zürich

Mobile Computing

Introduction

Stefan Mangold
stefan.mangold@inf.ethz.ch

Course Outline

Introduction

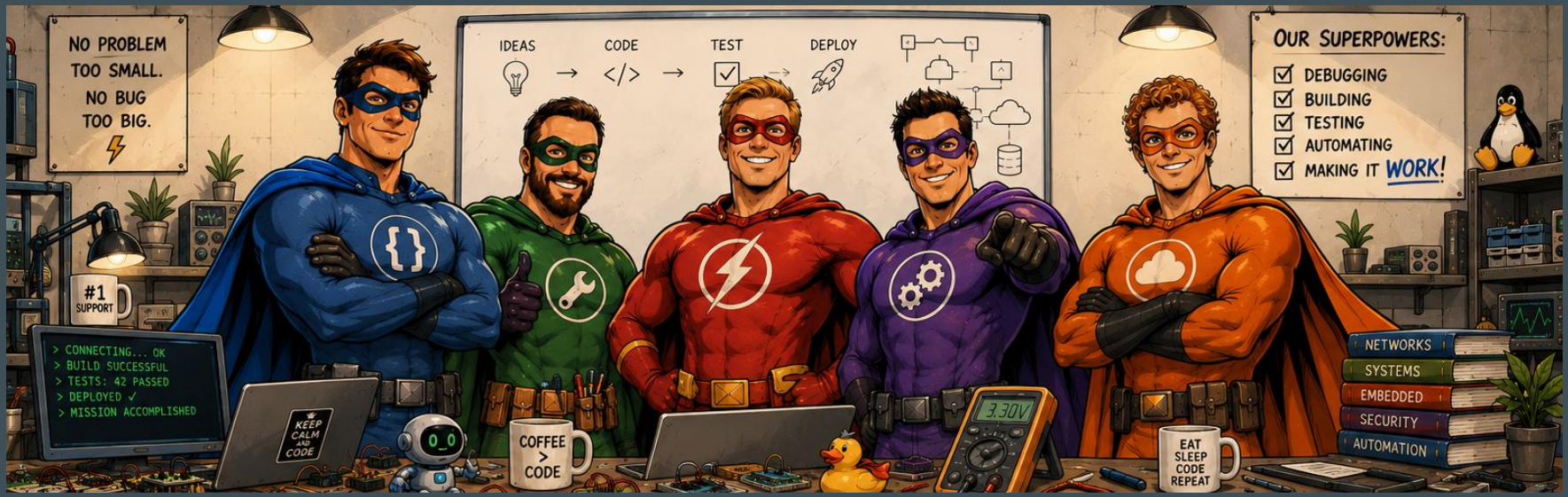
1. Data and Signals
2. Data Communication
3. Mobile Systems
4. Mobile Algorithms
5. Mobile AI
6. Cloud Computing

Tournament

Lecturer: Dr. Stefan Mangold

- Lovefield Wireless GmbH
- Lecturer at ETH Zurich
- Earlier
  - Ypsomed: Med Tech
  - Disney Research: Toys & Theme Parks
  - Swisscom: Telecom and Wireless
  - Philips Research: Wi-Fi, Defense
- EEng. @ RWTH Aachen, Germany

Teaching Assistants



For Efficient Participation



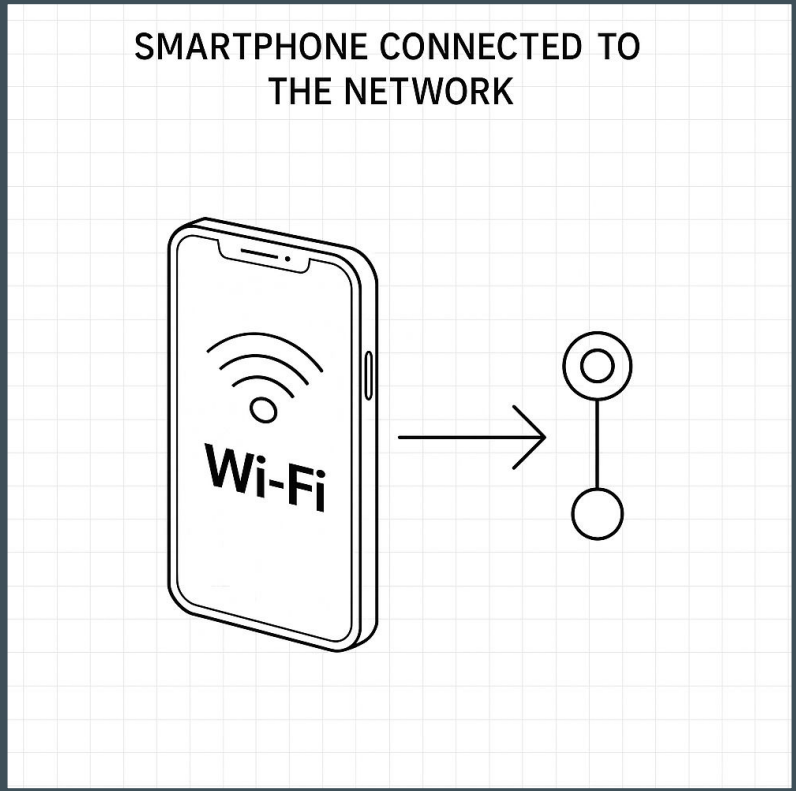
- Attend regularly and participate
- Interact with others and ask questions
- Stay persistent when challenges arise
- Start assignments early
- Ask for support or clarification whenever needed
- Share constructive feedback

Why Mobile Computing in this CAS?

- This course combines theory with practical insight into mobile computing
- Mobile computing enables decentralized and distributed systems
- Devices can communicate, adapt, and make decisions without central control
- On-device AI brings responsive, privacy-aware intelligence to the edge

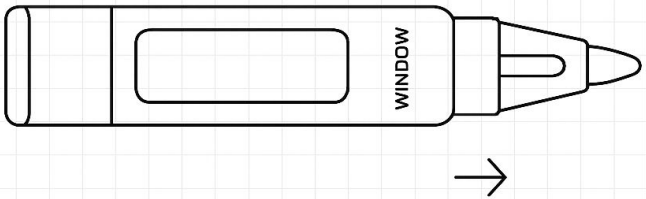
Mobile Computing Scenarios

- Non-stationary (“mobile”)
- Wireless communication
- Limited computation
- Limited energy
- Limited storage capacity
- Not fully reliable
- Low-cost, consumer device

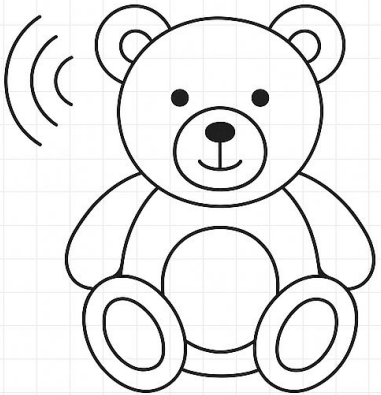


Mobile Computing Scenarios

MEDICAL
AUTOINJECTOR



INTERACTIVE
TOY

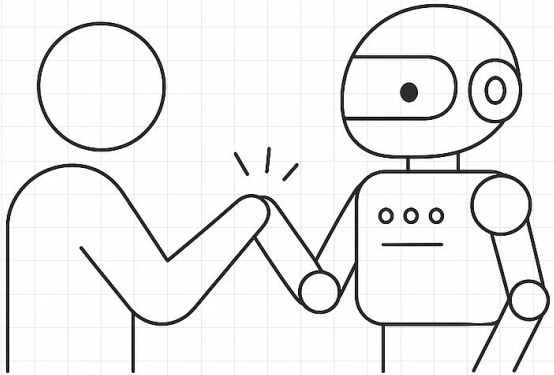


Mobile Computing Scenarios

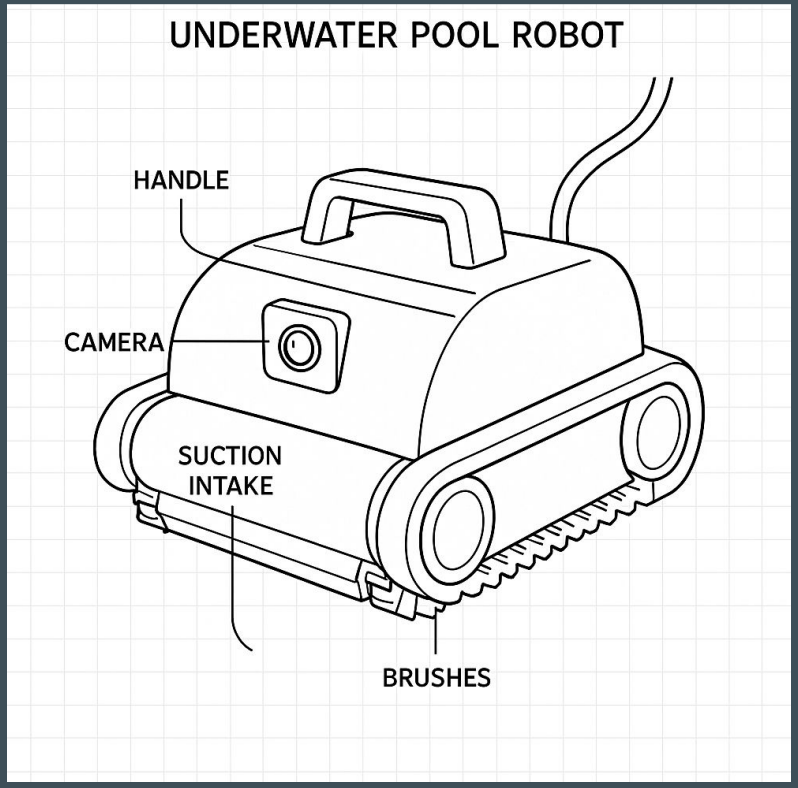
AUTONOMOUS
SELF-DRIVING CAR

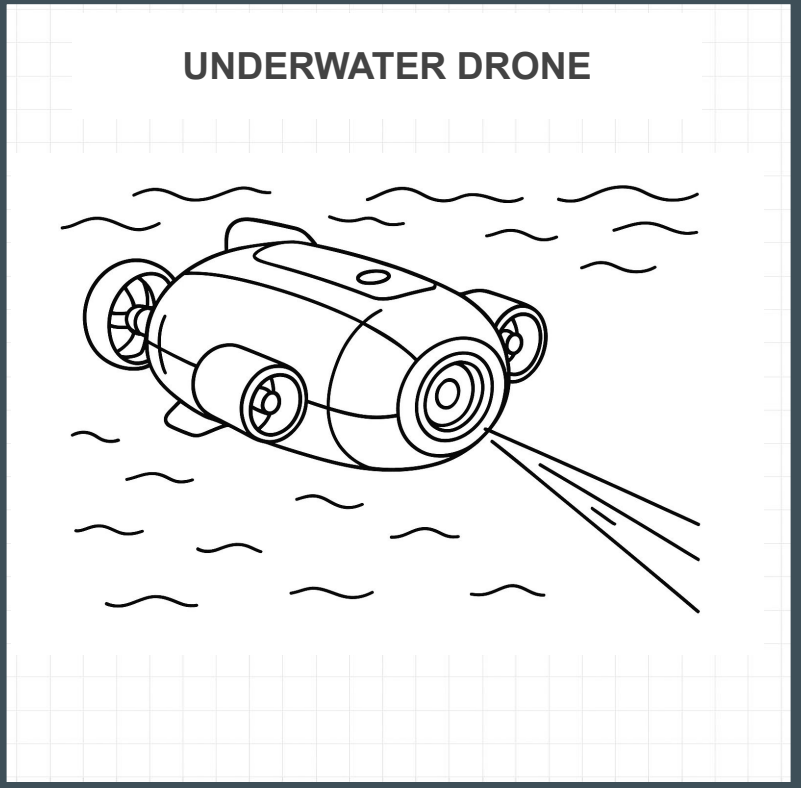


HUMAN-ROBOT
INTERACTION

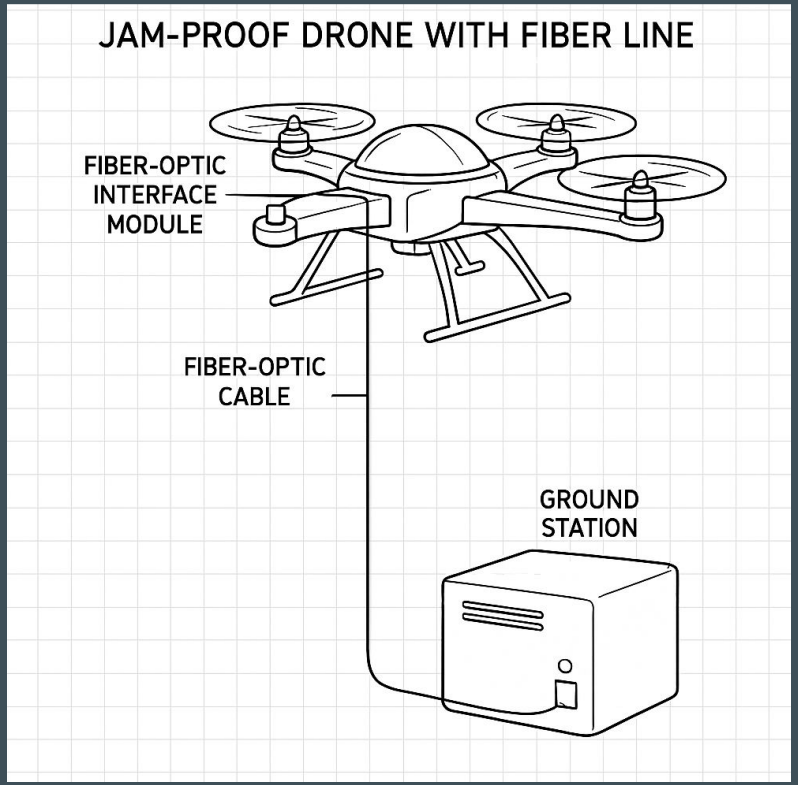


Mobile Computing Scenarios



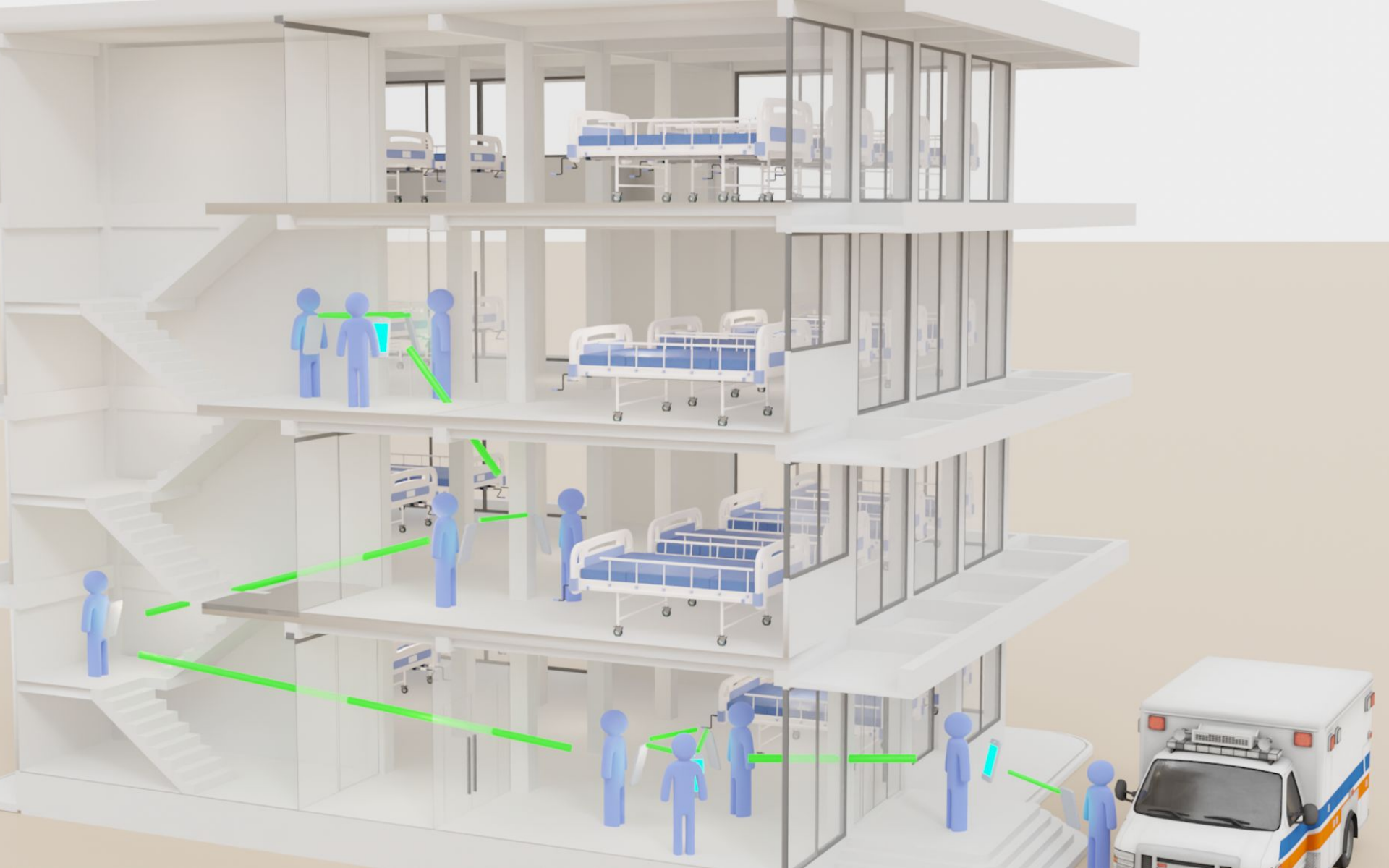


Mobile Computing Scenarios



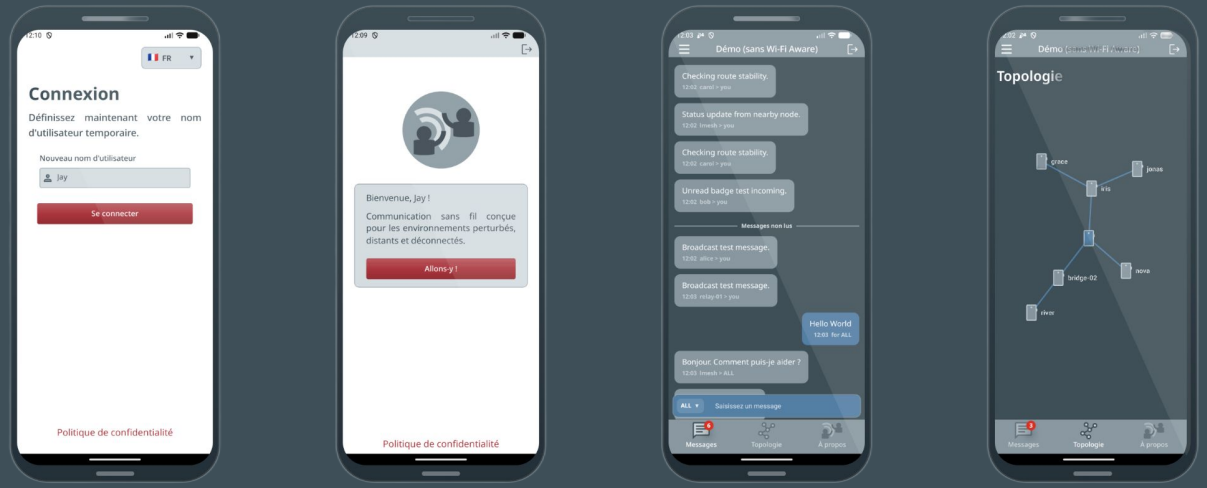






Prototype

- Experimental wireless communication when cellular and satellite networks are unavailable
- Messaging, local topology visualization, and simple onboarding for disrupted environments



Mobile and Cloud Computing

- Mobile devices provide anytime, anywhere access to data and services
- Cloud platforms supply scalable computing, storage, and coordination
- Edge processing keeps time-critical work close to users and devices
- Together, they create responsive and globally accessible applications

Mobile Computing and Artificial Intelligence

- Mobile AI turns sensor and usage data into context-aware decisions
- On-device models reduce latency, preserve privacy, and work offline
- **We will integrate small local language models into mobile algorithms**
- Local (unregulated) AI deployment raises legal, privacy, bias, and security questions

Course Overview and Objectives

- Part of the CAS Cloud & Mobile Computing
- Delivered over three weekends as two separate courses
- Continuing education in AI and Digital Technology
- Build practical knowledge of mobile systems, resource-constrained devices, and local LLMs
- Focus: 70% Mobile Computing, 25% Mobile AI, 5% Cloud Computing

Assessment and Grading

- Two independent courses, each graded separately
- Three assignments are provided and submitted as Jupyter Notebooks
- Submitted results determine your grades
- Some assignments may be completed in teams of two
- End-of-semester exam will be held only if needed

Information

- CAS Cloud & Mobile Computing [🔗]
- ETH Gitlab [🔗]
  - Lectures
  - Notebooks
  - Assignments
- Moodle [🔗]

Tools in This Course

- Course platforms
  - Moodle
  - GitLab
- Installed locally
  - Jupyter Notebooks
  - VS Code with Python, Java, Git, and AI extensions
- Electronic Hardware
  - Raspberry Pi
  - Arduino UNO

Use of Generative A.I.

- ETH Zurich advocates a proactive approach to the use of generative A.I. within educational contexts
- The use is generally permitted, within the boundaries of what ETH permits
- When using A.I., please clearly mark the generated content

Submission Guidelines

- Follow deadline stated in each assignment
- Work individually unless collaboration is explicitly permitted
- Submit completed notebooks
- Name files legi-number_0X; use a version suffix for revisions
- Mark AI-generated content and include all prompts
- See also: Submission-Guidelines.md

Git Basics

- Git records file changes as commits in a repository
- Working tree → staging area → repository history
- Run git status before updating or editing
- Clone once; use git pull to update a clean working tree
- Review changes with git diff; add and commit only intended files
- **See notebook: 00.5_Learning_Git_Basics.ipynb**

Vibe Coding Best Practices

- You remain accountable for your code
- Choose the simplest solution first
- Do not modify unrelated code
- State uncertainty and verify every change
- Never share secrets or personal data
- **See notebook: 00.4_Vibe_Coding_Best_Practices.ipynb**

00 Intro - Notebook Overview

- 00.1 - Check Course Environment
- 00.2 - Check Hardware Connectivity
- 00.3 - Scientific Literature and Research Integrity
- 00.4 - Vibe Coding Best Practices
- 00.5 - Learning Git Basics

Exercise: Check Course Environment

- Work through 00.1_Check_Course_Environment.ipynb
  - Set your local environment up for experiments and assignments
  - Fill in the answers to the questions
  - Save and upload the notebook file to Moodle
- Once you have uploaded your completed notebook, you should be ready for working with all the remaining notebooks in our course !

